

HealMesh — Product Requirements Document (PRD)

Status: Draft v1 **Owners:** Core (diagnosis/security) + Infra (cluster/execution) **Supersedes:** Original n8n + OpenClaw prototype ((prism) namespace). This PRD reflects the decision to build a direct-API core instead of routing diagnosis through n8n/OpenClaw.

1. Problem Statement

Kubernetes deployments fail for a small, recurring set of reasons — bad config, resource limits, image issues, rollout mistakes. When they fail, an on-call engineer has to read logs, guess at root cause, and apply a fix under time pressure, often without full context on the cluster's history. If the one engineer who knows the cluster well is unavailable, the whole team is slower to recover.

Existing tools (Datadog, Grafana, PagerDuty) detect and alert. None of them explain *why* something broke in plain language, tied to the specific failure, fast enough to matter during an incident.

2. Goal

Ship a system that watches a Kubernetes cluster, detects failures, and produces an accurate, plain-language root-cause diagnosis with a suggested fix — delivered to a human, in seconds, with zero risk to the cluster in its default mode. Remediation (the system actually acting on the cluster) is a later, explicitly-gated capability, not a v1 requirement.

3. Non-Goals (v1)

- No automatic remediation without human approval, ever, in any version of this product — approval is a permanent design invariant, not a v1 limitation to remove later at will (see TDD security invariants).
- No multi-cluster, multi-tenant SaaS in v1. Single cluster, single team.
- No custom LLM fine-tuning in v1. Use a hosted model via direct API.
- No dashboard in v1. Slack is the interface.
- No support for failure types outside the five canonical ones listed below, in v1.

4. Target User

An engineering team (starting with your own team/project, then a pilot user) running workloads on Kubernetes, who currently relies on a human reading logs and events to figure out why a pod or deployment failed.

5. Core User Story

"A pod in my cluster starts CrashLoopBackOff-ing. Within seconds, I get a Slack

message telling me the likely root cause in plain English, a confidence level, and the exact command I could run to fix it. I did not have to read a single log line myself to know where to start."

6. Scope by Phase

Phase 1 — Read-only diagnosis (the actual v1)

- Detect the five canonical failure types: `CrashLoopBackOff`, `OOMKilled`, `ImagePullBackOff`, failed rollout, resource quota exceeded.
- Produce a structured diagnosis: root cause, confidence, suggested command (as text, never executed).
- Deliver via Slack. No approval flow exists yet because nothing executes.
- Maintain an append-only incident/audit log of every diagnosis produced.

Phase 1.5 — Benchmark & demo

- A hand-built + real-incident benchmark set (30–50 cases minimum) with measured diagnosis accuracy, reported honestly.
- A recorded end-to-end demo: real failure injected → Slack diagnosis → manual fix → recovery confirmed.

Phase 2 — Gated remediation (single action type)

- One remediation action only: `SCALE` (replica count adjustment), the lowest-blast-radius action available.
- Requires explicit human approval via Slack before execution — no exceptions, no auto-approve mode in this phase.
- Post-execution health check with automatic rollback on failure.

Phase 3 — Expanded remediation + policy

- Additional action types (`PATCH`, `REDEPLOY`, `HELM_UPGRADE`) added one at a time, each gated by explicit approval by default.
- `HealPolicy` CRD lets a cluster operator configure per-namespace behavior (which actions are permitted, which namespaces are excluded).
- Dashboard replaces Slack-only interaction once incident volume justifies it.

Phase 4 — Platform expansion

- SDK for application/CI-level incident submission (not just cluster-native events).
- Multi-cluster support.
- Enterprise features (SSO, on-prem LLM option, blockchain-anchored audit log) — only once there's a real customer whose requirements are actually driving this.

7. Functional Requirements

ID	Requirement	Phase
FR-1	System detects the five canonical failure types via native Kubernetes event/API watching	1
FR-2	System produces a structured diagnosis (root cause, confidence, suggested command) for each detected failure	1
FR-3	Diagnosis is delivered to a Slack channel within a bounded time window (target: <30s from detection)	1
FR-4	Every incident and diagnosis is written to an append-only audit log	1
FR-5	System never executes any action against the cluster in Phase 1	1
FR-6	A benchmark dataset exists with measured, reported diagnosis accuracy per failure type	1.5
FR-7	System supports human-approved <code>SCALE</code> execution via Slack interaction	2
FR-8	Every execution is preceded by a snapshot and followed by an automated health check with rollback on failure	2
FR-9	System supports additional gated action types, each independently enabled	3
FR-10	<code>HealPolicy</code> CRD allows per-namespace configuration of allowed actions	3

8. Non-Functional Requirements

- **Security:** No component beyond the remediation executor has write access to the cluster, at any phase. The LLM's output is never executed directly; it is always parsed into a closed, typed enum before anything downstream sees it. (Full detail in TDD.)
- **Reliability:** A dead-man's-switch watchdog alerts if the detection service itself goes down.
- **Auditability:** Every diagnosis and every action (once remediation exists) is reconstructable from the audit log — who/what triggered it, what was diagnosed, what was suggested, what was approved, what happened.
- **Cost control:** LLM calls are rate-limited and log input is size-bounded, so an incident storm cannot cause runaway spend.
- **Honesty:** Accuracy claims are always backed by a measured benchmark, never asserted without evidence — internally and in any external-facing material.

9. Success Metrics

Metric	Phase 1 target
Diagnosis accuracy on benchmark set	≥ 80% correct root cause across the five failure types (report actual number regardless of where it lands)
Time from failure detection to Slack delivery	< 30 seconds, p95
False positive rate (diagnosis delivered for a non-failure)	As close to 0% as achievable; track and report
Engineer-reported usefulness (informal reaction/feedback in Slack)	Directional signal only in Phase 1, not a gate

10. Risks

Risk	Mitigation
LLM hallucinates a plausible-but-wrong diagnosis	Confidence scoring surfaced to the user; benchmark tracks accuracy per failure type so weak spots are visible, not hidden
Team over-invests in scope (dashboard, SDK, GTM) before core diagnosis is proven	PRD phases are sequential gates — Phase 2+ scope is explicitly blocked until Phase 1.5 benchmark and demo exist
Remediation trust concerns limit adoption	Addressed by design: remediation is opt-in, human-approved, single-action-at-a-time rollout, never framed as "fully autonomous"
Crowded competitive space (Komodor, k8sgpt, Robusta, observability vendors adding AI)	v1 differentiation is depth of diagnosis accuracy + honest, published benchmark — not breadth of features

11. Open Questions

- Which LLM provider/model for the diagnosis engine — finalize in TDD based on cost/latency/accuracy testing during Phase 0.
- Where does the benchmark dataset live long-term, and does it get published (useful for credibility, but consider whether real customer incident data can ever be included without anonymization work)?